

ARCA: Agente con Razonamiento, Conocimiento y Acción

Práctica final

Modelos Generativos Profundos · Máster en Inteligencia Artificial · ICAI · Curso 2026-2027

Repositorio: <https://github.com/kendrickcetina/dgm-arca>

1. Qué vais a construir

Durante el curso hemos visto cómo un modelo de lenguaje aprende a razonar, cómo se le enseña a usar herramientas, cómo se le conecta a una base de conocimiento y cómo todo eso se junta en un agente que actúa en bucle. En esta práctica lo construís vosotros, de principio a fin, **sobre un tema que elegís vosotros**.

La llamamos ARCA porque cada letra es una de las piezas: **R**azonamiento, **C**onocimiento, **A**cción, y el **A**gente que las une.

El tema es libre. Un asistente para preparar unas oposiciones, un agente que lee la normativa de un ayuntamiento, un tutor de matemáticas, un copiloto fiscal para autónomos, un entrenador de programación competitiva. Lo que queráis, con una condición: que resuelva un problema a alguien de verdad, aunque ese alguien seáis vosotros. Si al terminar podéis enseñar el repositorio en una entrevista de trabajo, habréis hecho bien el trabajo.

Sé que vais a programar con ayuda de agentes de código, y no os lo voy a prohibir. Precisamente por eso lo que puntúa no es escribir código, sino **decidir qué construir, saber si funciona y explicar por qué**. Un agente de código os escribe un bucle GRPO en dos minutos; lo que no puede hacer es elegir la tarea verificable de vuestro dominio, diseñar el verificador, construir el corpus ni interpretar por qué la recompensa se estanca.

Tres condiciones que el tema debe cumplir. (1) Alguna tarea con **respuesta comprobable automáticamente**: un número, una fecha, un código que pasa tests, una etiqueta. Sin verificador no hay recompensa y sin recompensa no hay refuerzo. (2) **Herramientas con sentido** en ese dominio: algo que consultar, algo que calcular, algo que hacer. (3) **Documentación real** que podáis recopilar. Sin corpus no hay RAG.

2. Equipos, ritmo y forma de trabajar

Equipos de tres personas. Repositorio compartido, un solo entregable por equipo, y en la defensa oral pregunto a cualquiera de los tres sobre cualquier parte del código.

La práctica se construye **por capas, y se prueba desde el primer día**. No esperéis a tenerlo todo: cada fase termina en un endpoint que funciona por sí solo, así que podéis levantar la API con la fase 1 hecha y las otras tres devolviendo «pendiente».

Fase	Carpeta	Endpoint	Cuándo
0. Propuesta	docs/	—	Primera semana, bloqueante
1. Razonamiento	rlm/	/reasoning	Empezamos el lunes 21 de septiembre
2. Herramientas	tool_use/	/tools	A continuación
3. Conocimiento	rag/	/rag	A continuación
4. Agente	agent/	/agent	Cierra el proyecto

El orden no es casual: primero el modelo aprende a pensar, luego le damos manos, luego memoria externa, y al final lo ponemos a trabajar solo usando todo lo anterior. En la fase 4 reutilizaréis el modelo de la fase 1, las herramientas de la fase 2 y el retriever de la fase 3. Pensad en el conjunto desde el principio.

Las fechas exactas de cada entrega las fijamos en clase. Cada README de carpeta tiene el detalle completo; este documento es el resumen.

3. Fase 0: la propuesta

Antes de escribir código, una página por equipo. No es burocracia: es la parte que más determina vuestra nota, porque un tema bien elegido hace que todo lo demás fluya.

Plantilla en `docs/00_propuesta.md`. Quiero leer: el tema y el usuario; diez preguntas reales que ese usuario haría; **qué tarea es verificable y cómo la verificaríais**; qué herramientas harían falta; de dónde sale el corpus; y qué puede salir mal.

Si no tenéis tema, en `docs/temas_ejemplo.md` hay trece desarrollados (farmacia, legal, finanzas, desarrollo, social, marketing, ciencia) con su tarea verificable, sus herramientas y su corpus. Son para inspirar, no para elegir de una lista.

Hasta que la propuesta esté aprobada, no empecéis la fase 1. Cuando la apruebe os devolveré por escrito qué se evalúa exactamente en vuestro tema.

4. Fase 1: Razonamiento (`r1m/`)

Convertir un modelo de lenguaje en un modelo que piensa: que genere su razonamiento entre `<think>` y `</think>` y después la respuesta entre `<answer>` y `</answer>`. Reproducís, a pequeña escala y sobre vuestro dominio, el camino de DeepSeek-R1: arranque en frío con SFT y después refuerzo con recompensas verificables.

Mínimos de la fase 1

1. **Dataset verificable propio** (cientos de problemas) y un **verificador determinista** en `verifier.py`, con tests de sus casos límite.
2. **Destilación**: generar trazas con un modelo profesor, filtrarlas con el verificador y quedarse con las correctas. Reportad la tasa de aceptación.
3. **SFT** con LoRA sobre las trazas verificadas.
4. **GRPO** con al menos tres recompensas: formato, exactitud y una **propia de vuestro dominio** que tendréis que justificar.
5. **Un paso de GRPO implementado a mano** (ventajas, ratio de políticas, recorte, KL) con sus tests en verde.
6. **Evaluación**: `pass@1` en test para modelo base, tras SFT y tras GRPO; curvas de recompensa y de longitud; análisis de cinco fallos concretos.

Ficheros. Ya hechos: `rewards.py` (recompensas de formato y exactitud, con tests), `data.py` (prompt de sistema de R1-Zero y carga de datos), `generate_problems.py` (generador de problemas de ejemplo, ejecutable), `inference.py` (detrás del endpoint). Vuestros: `verifier.py`, `distill.py`, `train_sft.py`, `train_grpo.py`, `grpo_step.py`, `evaluate.py`.

¿De dónde salen cientos de problemas? No de anotarlos a mano. Las trazas de razonamiento las genera el modelo profesor y las filtra vuestro verificador. Y los pares (enunciado, respuesta) se consiguen con un generador programático, minando datos estructurados, o con benchmarks públicos del dominio. Está todo en `docs/datasets.md`, con tamaños de referencia y los errores típicos.

Antes de empezar, corred el entrenamiento de prueba: `uv run arca-smoke`. Es GRPO de verdad en miniatura sobre GSM8K, con las mismas recompensas que usaréis. En diez minutos veréis la recompensa de formato subir. Detalles en `smoke/README.md` y `docs/dgx.md`.

5. Fase 2: Herramientas (`tool_use/`)

Un modelo solo produce texto: no puede consultar una API ni ejecutar código. Tool use es dárselo. El modelo genera una llamada en un formato reconocible, la aplicación la detecta, la ejecuta y le devuelve el resultado. Los cinco pasos que vimos en clase, implementados a mano.

Mínimos de la fase 2

1. **Tres herramientas reales** de vuestro dominio: una que consulte una API HTTP externa de verdad, una que calcule o ejecute, y una que **actúe** con efecto observable (escribir un fichero, crear un evento, enviar a un webhook de pruebas).
2. **Definiciones como JSON Schema** generadas desde Pydantic, con los argumentos validados antes de ejecutar; un error de validación vuelve al modelo como observación.
3. **El flujo de cinco pasos escrito por vosotros**, sin frameworks de agentes, con soporte para llamadas encadenadas (multi-step) y varias llamadas en una generación (paralelas).
4. **Banco de pruebas de treinta casos** con la herramienta y los argumentos esperados, incluyendo casos que **no** necesitan herramienta. Métricas: precisión de selección y de argumentos. Comparad el modelo con y sin herramientas.

Ficheros. Ya hechos: `registry.py` (convierte funciones tipadas en definiciones y valida), `parser.py` (detecta `<tool_call>`), `tools.py` (dos herramientas de ejemplo), `evaluate.py` (métricas). Vuestros: vuestras herramientas en `tools.py`, `executor.py` y el banco en `bench/`.

En esta fase no se usa smolagents. Quiero ver vuestro bucle. En la fase 4 sí lo permito, como contraste.

6. Fase 3: Conocimiento (rag/)

El conocimiento del modelo se congela en su fecha de corte, y meterle documentos enteros en el contexto no funciona. RAG es la alternativa: trocear el corpus, indexarlo, y recuperar solo los fragmentos relevantes en el momento de la pregunta.

Mínimos de la fase 3

1. **Corpus real** de vuestro dominio con ingesta reproducible (del orden de decenas de documentos o varios cientos de fragmentos), y su licencia documentada.
2. **Dos estrategias de troceado comparadas** con números: la recursiva y la semántica basada en embeddings de frase.
3. **Embeddings instruction-aware** (Qwen3-Embedding), almacén vectorial, **BM25** y **recuperación híbrida** con un peso λ ajustable.
4. **Conjunto dorado de cincuenta preguntas** anotadas con el fragmento que las responde, y **Recall@k** y **MRR** para los tres retrievers. El λ se elige con datos, no a ojo.
5. **Respuestas con citas** a los fragmentos usados, y una comprobación automática de que las citas existen.

Ficheros. Ya hechos: `ingest.py` (lee PDF, Markdown, texto, HTML), `chunking.py` (troceado recursivo y estadísticas), `embed.py` (Qwen3-Embedding con instrucción), `retriever.py` (BM25), `evaluate.py` (Recall@k y MRR), `generate.py` (prompt aumentado y extracción de citas). Vuestros: el troceado semántico, el retriever denso, el híbrido, el corpus en `corpus/` y el conjunto dorado en `gold/`.

El trabajo de verdad de esta fase es el conjunto dorado. Se escribe leyendo el corpus, y es lo que os permite decir qué retriever es mejor en vuestro dominio en lugar de suponerlo.

7. Fase 4: Agente (agent/)

Todo junto en un bucle: piensa, actúa, observa, y vuelve a pensar hasta responder. Es el patrón ReAct, y es donde se juntan las tres fases anteriores.

Mínimos de la fase 4

1. **Bucle ReAct propio** con `final_answer`, límite de pasos, timeouts por herramienta y recuperación de errores. El retriever de la fase 3 entra como la herramienta `search_knowledge_base`.
2. **Cerebro intercambiable** y comparado: modelo base, vuestro modelo de la fase 1, y un modelo con razonamiento nativo. ¿Ayuda de verdad haber entrenado el modelo?
3. **Memoria** multi-turno que se resume automáticamente al superar un presupuesto de tokens.
4. **Banco de veinte tareas** que requieran encadenar al menos dos herramientas, con tasa de éxito, número medio de pasos y coste en tokens. Comparad vuestro agente JSON con un agente de código.
5. **Traza visible**: cada ejecución devuelve todos los pasos, y una interfaz mínima los muestra.

Ficheros. Ya hechos: `prompts.py` (prompt ReAct), `memory.py` (memoria con presupuesto), `evaluate.py` (métricas por cerebro), `ui.py` (visor de trazas). Vuestros: `react_agent.py`, la política de memoria y el banco en `bench/`.

8. Todo se prueba desde la API

La corrección se hace llamando a vuestra API. No leo vuestros notebooks para saber si funciona: lanzo peticiones. Por eso los contratos de entrada y salida están fijados en `api/schemas.py`. Podéis añadir campos opcionales a las respuestas, pero no renombrar ni quitar los que ya están.

Endpoint	Envío	Devuelve
GET /health	—	Estado de cada fase: ready, pending o error
POST /reasoning	question, expected_answer?	thinking, answer, has_valid_format, veredicto del verificador
POST /tools	query, max_turns?	answer y tool_calls[] con nombre, argumentos, resultado y latencia
POST /rag	question, top_k?, retriever?	answer, chunks[] con id, fuente y puntuación, y citations[]
POST /agent	task, max_steps?, brain?	final_answer y steps[] con pensamiento, acción y observación

Cómo funciona mientras desarrolláis. La API arranca siempre, aunque no hayáis hecho nada: una fase sin implementar responde 501 con su nombre y un mensaje, y /health os dice qué está listo. Así entregáis por capas y probáis cada fase en cuanto la terminéis.

Un ejemplo de lo que espero ver.

```
$ curl -s localhost:8000/health
{"status": "ok", "team": "equipo-07", "domain": "fiscalidad de autonomos",
 "phases": {"reasoning": {"status": "ready"}, "tools": {"status": "pending"}, ...}}

$ curl -s -X POST localhost:8000/reasoning -H 'Content-Type: application/json' \
  -d '{"question": "...", "expected_answer": "120"}'
{"thinking": "Sea C la capacidad del deposito...", "answer": "120",
 "has_valid_format": true, "tokens_generated": 214,
 "verifier": {"is_correct": true, "predicted": "120", "expected": "120"}}
```

Para la corrección. La API tiene que levantar con `docker compose up api` en una máquina limpia. La exponéis con `ngrok` (`ngrok http 8000`) y me pasáis la URL. Antes de pasármela,

probad los cuatro endpoints desde `/docs` y comprobad que `/health` los marca como ready. Rellenad también `ARCA_TEAM` y `ARCA_DOMAIN` en vuestro `.env`: salen en `/health` y me dicen a quién estoy corrigiendo. Detalles en `api/README.md`.

9. Cómo se evalúa

Los cuatro criterios se aplican **a cada fase por separado**, y cada fase pesa lo mismo en la nota final.

Criterio	Peso	Qué miro
Implementación correcta	30 %	El endpoint responde y hace lo que dice. Lo compruebo con un script que lanza peticiones, algunas parecidas a vuestros bancos de pruebas y otras que no habéis visto.
Completitud	30 %	Los mínimos de la fase están hechos. Cada pieza que falte resta.
Interpretación y explicación	30 %	Curvas y métricas explicadas, no solo pintadas. Comparaciones y ablaciones. Análisis de fallos. Cuaderno de experimentos honesto. Y la defensa oral .
Limpieza y calidad	10 %	Sin marcadores pendientes. <code>uv sync</code> reproduce el entorno, los tests pasan, Docker levanta, README de portfolio, sin secretos en el repositorio.

La diferencia entre un trabajo correcto y uno excelente está en el tercer criterio. Quiero que sepáis por qué vuestra recompensa es como es, qué hace el recorte en GRPO, por qué elegisteis ese λ y qué pasó cuando le quitasteis el modelo de razonamiento al agente. Si el código lo escribió un agente y no sabéis explicarlo, se nota.

Cada fase se entrega con su parte del cuaderno de experimentos (`docs/EXPERIMENTS_plantilla.md`), fechado: una entrada que diga «probamos esto, no funcionó, creemos que por aquello» vale más que diez que digan «todo bien». La rúbrica completa está en `docs/rubrica.md`.

10. La entrega final

Cuando terminéis las cuatro fases tendréis un sistema completo. Tratadlo como un producto:

- Repositorio público con README de portfolio: qué hace, para quién, una demo, un diagrama, la tabla de métricas de las cuatro fases y cómo levantarlo con una orden.
- El cuaderno de experimentos completo y un informe técnico de diez páginas como máximo (plantilla en `docs/informe_plantilla.md`).
- **Defensa oral** de diez minutos por equipo: tres de demo en vivo y el resto de preguntas sobre vuestro código y vuestros resultados.

11. Por dónde empezar

1. Entrad en <https://github.com/kendrickcetina/dgm-arca> y pulsad **Use this template** para crear el repositorio de vuestro equipo. Si no habéis usado GitHub antes, o no sabéis cómo hacer push desde la DGX, está paso a paso en `docs/github.md`.
2. Montad el entorno: `uv sync -extra train` y `uv run pytest`. Todo en verde antes de tocar nada.
3. En la DGX: sesión de code-server desde <https://hpc.comillas.edu>, `source smoke/dgx_env.sh`, `uv run arca-check-gpu` y `uv run arca-smoke`. Guía completa, con la GPU de 16 GB y las sesiones de 24 horas: `docs/dgx.md`.
4. Escribid la propuesta (`docs/00_propuesta.md`) y empezad por la sección de la tarea verificable. Si esa sección se escribe sola, tenéis tema.

Un último consejo. Empezad por el verificador y por el conjunto dorado, no por el modelo: lo que no se puede medir no se puede mejorar. Y haced `git push` al terminar cada sesión, que la DGX no tiene copias de seguridad.